Parallel and Distributed Systems: Paradigms and Models
Project 1 (MergeSort) Report

Davide Marchi (602476)
d.marchi5@studenti.unipi.it

August 2025

Contents

1 Introduction

I present a complete study of a distributed out-of-core MergeSort. The study proceeds in four steps. First, I describe the problem and the data layout. Second, I implement four solutions that share a common I/O pipeline while differing in how they exploit parallelism: a sequential baseline, an OpenMP task based version, a FastFlow farm based version, and a multi node hybrid MPI solution that reuses the single node core. Third, I evaluate these versions under controlled workloads and hardware configurations, measuring runtime, speedup, and efficiency. Finally, I discuss the results and propose an approximate cost model for the distributed solution.

All implementations work on files that are potentially larger than available RAM. Each program first scans the unsorted file to build an in memory index of keys and positions and lengths, then sorts this index, then rewrites a new file in key order by fetching payloads at the recorded positions. To keep memory usage proportional to the number of records, the in memory structure stores only keys, offsets, and lengths. The write phase can be highly variable on a shared cluster, so the primary performance analysis focuses on the reading and sorting stages. The full pipeline remains functionally correct and the final output is validated.

The objectives are:

- demonstrate correctness of all variants by verifying that keys are globally sorted in the *written* output file and that records are preserved;
- quantify single node scalability with strong and weak scaling for OpenMP and FastFlow;
- quantify multi node scalability for the MPI hybrid, using one rank per node and varying threads per rank;
- interpret bottlenecks that arise from memory access, tasking overheads, and communication.

Structure. Section 2 defines the problem and data model. Section 3 discusses memory and I/O considerations. Section 4 presents single node implementations and their evaluation. Section 5 presents the multi node implementation and its evaluation. Section 6 proposes an approximate cost model and discusses bottlenecks and overlap.

2 Problem description

2.1 Data model and constraints

The input is a large binary file that stores an array of records. Each record consists of a sortable key, a length field, and a payload. The payload size can be large compared to the key, and the file may exceed available RAM, which motivates an out-of-core strategy.

I use the following logical layout for each **Record**:

- **key**: 64 bit unsigned integer used for ordering;
- **len**: 32 bit unsigned integer that gives the payload length in bytes, with len $\leq P$ for a dataset parameter P;
- **payload**: a byte array of length len.

Sorting records in place is impractical when the file is larger than RAM. Instead, I construct an auxiliary in memory index. Each entry is an **IndexRec** that stores only what is needed to order and later locate the record:

- **key**: 64 bit unsigned integer, copied from the record;
- **pos**: byte offset in the unsorted file where the record begins;
- **len**: 32 bit unsigned integer, copied from the record, used to stream exactly len bytes during rewrite.

The index has length N, one entry per record. Its memory footprint is $O(N)$ and is independent of the maximum payload size P. This design keeps memory predictable and minimizes disk traffic during random access.

Correctness means that the output file contains exactly the input records in nondecreasing key order. I validate this by scanning the *written* output file from disk, checking that keys are ordered and that the record count matches the input.

2.2 Pipeline, distributed setting, and metrics

The pipeline is the same across all variants:

1. **Index build.** Scan the unsorted file, read `key` and `len` per record, and fill the `IndexRec` array with {key, pos, len}. File access uses memory mapping to support efficient random access patterns and to avoid large user space buffers.
2. **Index sort.** Sort the `IndexRec` array by key. The sequential baseline uses `std::sort`. The parallel versions use a task based mergesort with a base case threshold that delegates small subarrays to `std::sort`.
3. **Rewrite.** Create the output file, then for each entry in sorted order fetch the record at `pos` from the unsorted file and write it to the next position in the output file, streaming exactly `len` bytes for the payload. This preserves the original payloads without storing them all in memory.
4. **Validation.** Scan the output file to confirm nondecreasing key order and that the record count is unchanged.

In the multi node variant, I run one MPI rank per node. A designated rank reads the file and builds the global index, then partitions that index into contiguous slices and sends one slice to each rank. Each rank sorts its slice locally. The globally sorted order is

obtained through a sequence of pairwise merges across ranks. The final rank rewrites the output file and performs validation. Communication is limited to index data and merged index segments, which keeps memory usage predictable and decoupled from payload size.

The primary metrics are total time for index build plus sort, speedup $S_p = T_1/T_p$, and efficiency $E_p = S_p/p$. For single node studies, p is the thread count. For distributed studies, p is the product of nodes and threads if noted, or the number of threads per rank when speedup is plotted against nodes. Strong scaling holds N fixed while varying p. Weak scaling grows N proportionally to p.

3 Memory and I/O considerations

My design keeps memory usage proportional to the number of records rather than to the maximum payload size or to an arbitrary buffer. I maintain an in memory index of length N, where each entry stores only the information required to order and later locate a record: $\{\texttt{key}, \texttt{pos}, \texttt{len}\}$. The footprint is therefore $O(N)$ with a small constant factor that depends on the sizes of these fields, and it does not depend on the maximum payload parameter P. During rewrite I stream each payload directly from the unsorted file into the output without caching all payloads in RAM.

3.1 Why I use memory mapping

I use `mmap` for both reading and writing. This choice avoids explicit user space I/O buffers and lets the operating system handle paging and readahead. It is well suited to the two access patterns in this project:

- During index build, I scan the file record by record and read only `key` and `len`. Mapping the file allows the kernel to fault in pages efficiently and to reuse them across passes when the page cache is warm.
- During rewrite, I must fetch payloads at the original positions and emit them in sorted order. This requires many random reads of the source and a sequential write to the destination. With `mmap` I can copy exactly `len` bytes per record from the mapped source region to the mapped destination region without building large staging buffers.

This approach keeps per record work bounded and predictable, and it aligns with the constraint that the file may be much larger than available RAM.

3.2 Single threaded disk access

On the shared cluster, attempts to parallelize disk access did not produce stable gains. Running multiple threads that read or write concurrently tended to increase queue contention and amplify interference from other users on the same storage. I therefore adopt a conservative policy:

1. I perform file I/O with a single thread at a time.
2. I exclude the rewrite phase from the primary performance graphs and focus the analysis on index build plus sorting.

This policy isolates the compute and indexing behavior of the implementations from storage noise and makes cross run comparisons meaningful.

3.3 Variability of the write phase

The write phase is highly volatile on the target system. In three executions of the same configuration, the time to rewrite the sorted file differed by almost an order of magnitude. Table 1 reports the raw measurements.

Table 1: Observed variability of the rewrite phase for identical runs

Run	Write time (s)	Notes
1	68.97	light system load
2	264.69	moderate contention
3	503.60	heavy contention

Given this spread, including rewrite time in the main figures would mask the effects I want to study. I still execute the full pipeline in every run and I always validate correctness on the *written* output file from disk, but I report index build plus sorting as the primary metric in Sections 4 and 5.

4 Single node implementations

All single node variants share the same three stage pipeline: I first scan the file to build an in memory index of $\{\texttt{key}, \texttt{pos}, \texttt{len}\}$ entries, then I sort this index, then I rewrite the output file by streaming each payload from the original position to its new position. Using the `IndexRec` vector keeps memory proportional to the number of records N, independent of the maximum payload size P. For the parallel variants I use a task based mergesort with a base case threshold of 10 000 elements that falls back to `std::sort` to avoid unnecessary tasking overhead.

4.1 Sequential baseline

The sequential program performs no overlap. It scans the file and fills the `IndexRec` vector, sorts the entire vector with `std::sort`, then rewrites the output file by copying exactly `len` bytes for each record from the source mapping at `pos`. This run is my reference T_1 for speedup.

4.2 FastFlow implementation

I use a single FastFlow farm. With n threads, the farm consists of one emitter and $n - 1$ workers.

- **Single thread fallback.** If only one thread is available, I bypass the farm and call `std::sort` on the whole index.
- **Tasking strategy.** The emitter first schedules the index build as the initial task. It then generates all mergesort tasks in a recursive manner. When a subarray size reaches the base case of 10 000, the corresponding leaf task is enqueued immediately and executed with `std::sort`.
- **Readiness gating.** To start sorting as soon as possible, leaf tasks coordinate with a `ProgressGate`. Each leaf knows its $[L, R)$ range of index entries and sleeps until the index builder signals that the required portion has been filled. The reading worker wakes up any waiting leaves every 10 000 records in the best case.

This design allows the workers to overlap sorting of ready chunks with the ongoing index build. The emitter cost means that with $n = 2$ the farm has only one active worker, which directly affects that specific case in the results.

4.3 OpenMP implementation

I implement a task based top down mergesort over the `IndexRec` vector while the index is being built.

- **Scheduling.** Inside a single OpenMP parallel region, I spawn one task that builds the index and, in parallel, I generate the mergesort task tree with `#pragma omp task` and `taskgroup`. As in the FastFlow version, any subarray of size at most 10 000 is handled by `std::sort`.
- **Readiness gating.** Leaf tasks check the `ProgressGate` to ensure the needed slice of the index has been produced before proceeding.
- **Single thread behavior.** When there is exactly one OpenMP thread, I enforce a `taskwait` after the index build task and *before* spawning the merge tasks. This prevents the single available thread from being occupied by merge tasks that would block waiting for data the reader cannot produce concurrently.

4.4 Performance evaluation

I evaluate index build plus sort time for the sequential, FastFlow, and OpenMP implementations over multiple input sizes, payload maxima, and thread counts. For each configuration I aggregate repeated runs and report medians.

4.4.1 Execution times

I report the total time

$$T \;=\; T_{\text{build-index}} \;+\; T_{\text{sort-index}}\,.$$

This excludes the rewrite phase as discussed in Section 3.

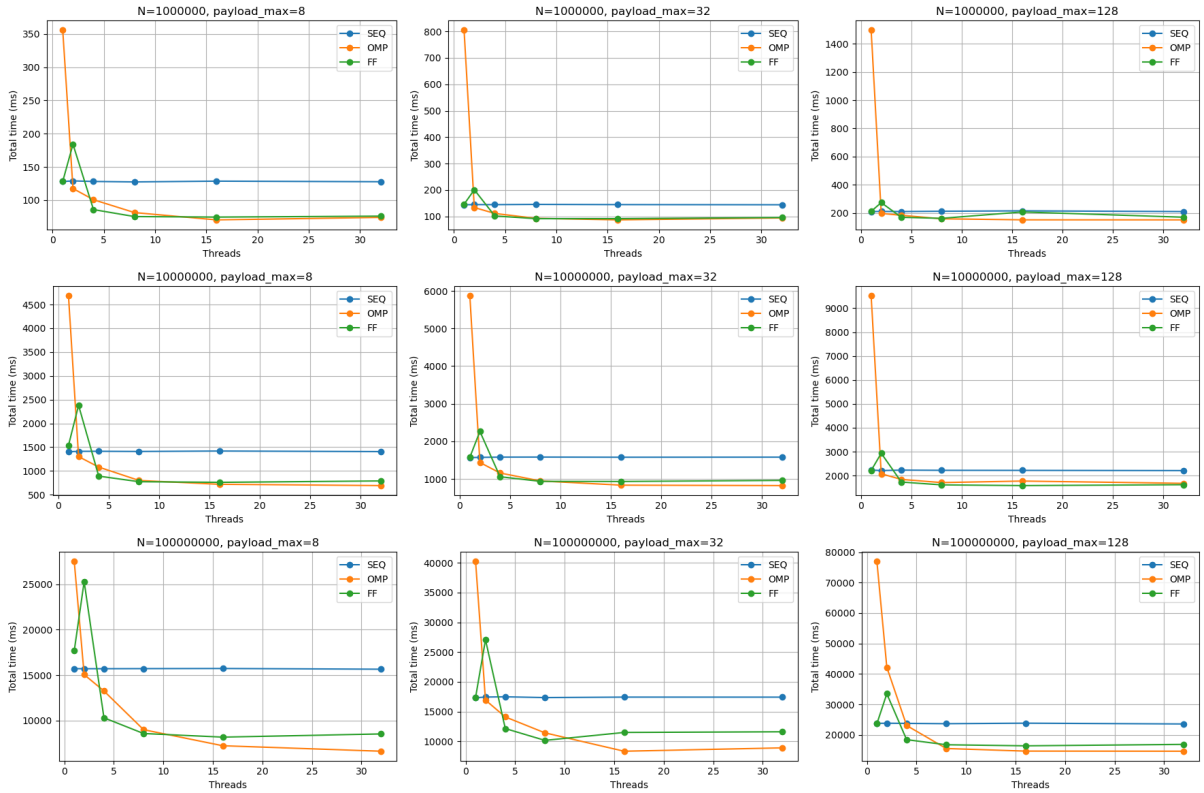

Figure 1: Execution time T for index build plus sort across a 3×3 grid of $(\text{records}, \text{payload}_{\max})$.

4.4.2 Absolute speedup

Absolute speedup is defined against the *best sequential* time for the same dataset settings:

$$S_{\text{abs}} \;=\; \frac{T^{*}_{\text{seq}}}{T_{\text{variant}}}\,,$$

where T^{*}_{seq} is the fastest observed sequential time for that $(N, \text{payload}_{\max})$ and T_{variant} is the time of the FastFlow or OpenMP run.

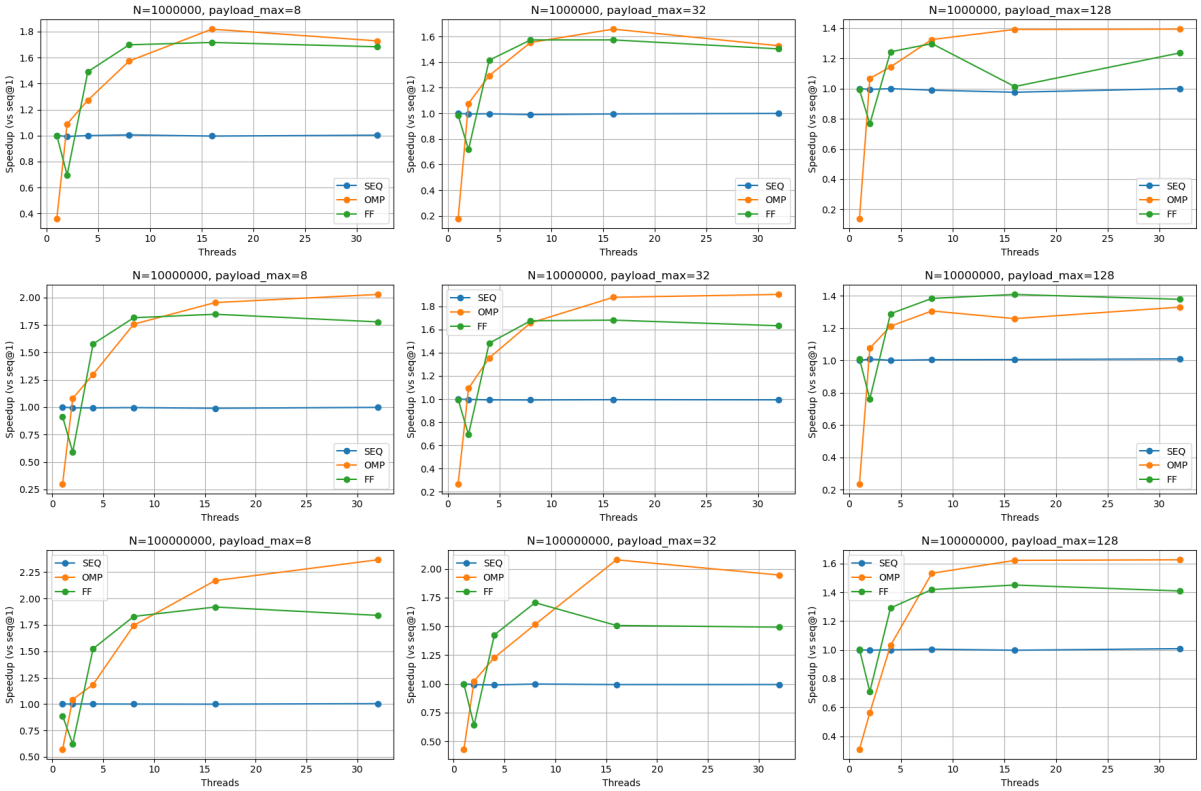

Figure 2: Absolute speedup S_{abs} relative to the best sequential time on the same 3×3 grid.

Absolute efficiency. To normalize for the number of threads, I also report *absolute efficiency*

$$E_{\text{abs}} = \frac{S_{\text{abs}}}{p} = \frac{T^*_{\text{seq}}}{p \cdot T_{\text{variant}}},$$

where p is the thread count of the parallel run (for FastFlow, p is the total threads including the emitter). Values near 1 indicate near ideal utilization at that problem size; lower values reflect overheads or memory/IO bottlenecks.

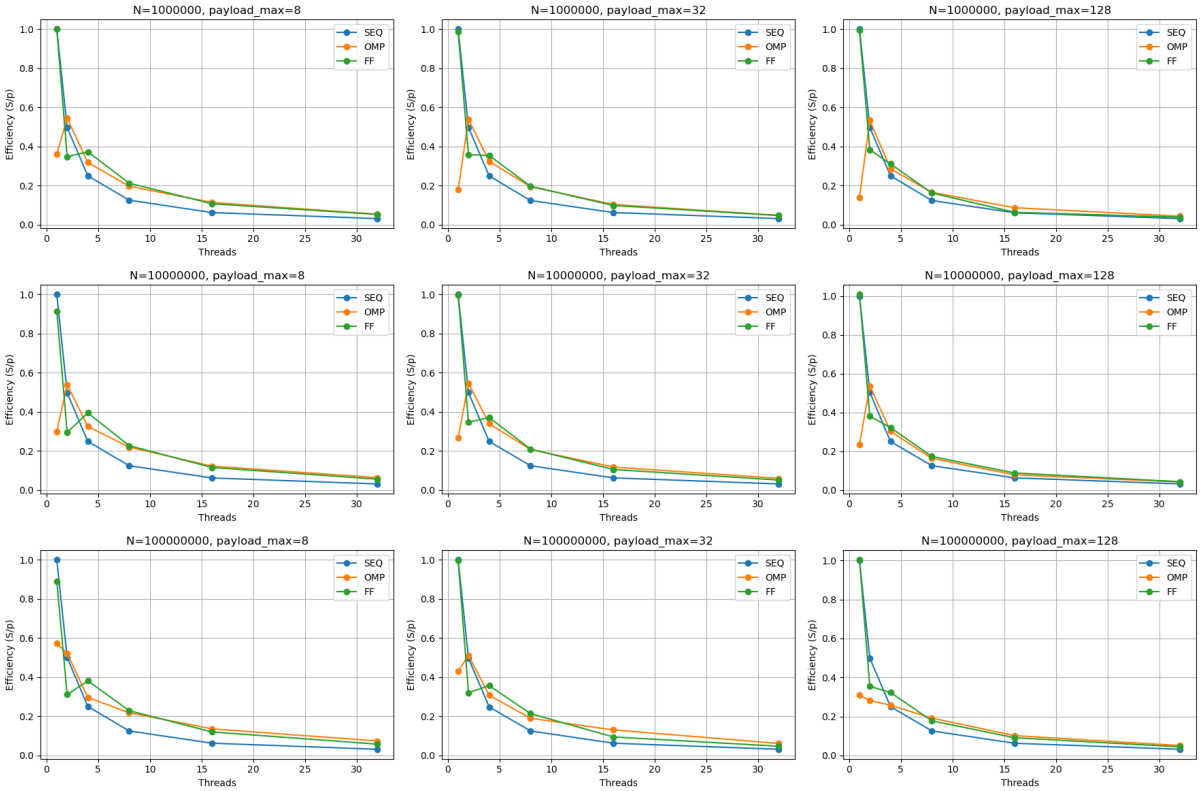

Figure 3: Absolute efficiency $E_{\text{abs}} = S_{\text{abs}}/p$ on the same 3×3 grid of (records, $\text{payload}_{\max}$).

4.4.3 Strong scalability

For strong scaling I fix N and vary the thread count p. I use relative speedup with respect to the *same* parallel implementation at one thread:

$$S_{\text{strong}}(p) \;=\; \frac{T_{\text{par}}(1, N)}{T_{\text{par}}(p, N)}\,.$$

The figure shows time, relative speedup, and efficiency panels for completeness.

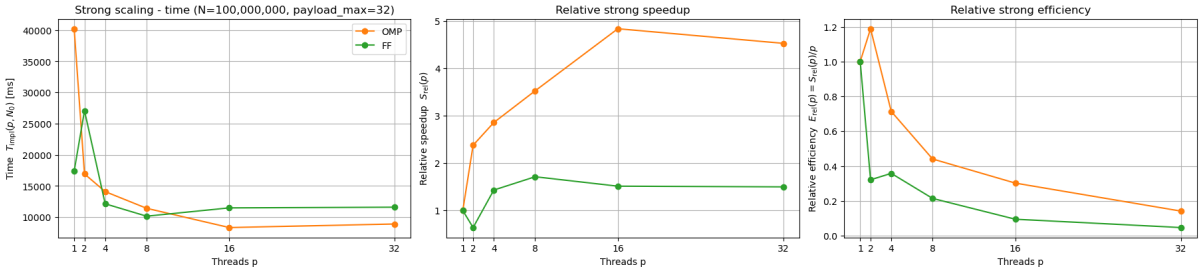

Figure 4: Strong scaling at $N = 100{,}000{,}000$ and $\text{payload}_{\max} = 32$: time, relative speedup $S_{\text{strong}}(p)$, and efficiency.

4.4.4 Weak scalability

For weak scaling I increase the number of elements with the thread count to reflect the $O(N \log N)$ cost of sorting. I choose N_p so that $N_p \log_2 N_p$ grows approximately linearly

with p, and I use relative speedup with respect to the parallel implementation at one thread:

$$S_{\text{weak}}(p) \;=\; \frac{T_{\text{par}}(1, N_1)}{T_{\text{par}}(p, N_p)}\,.$$

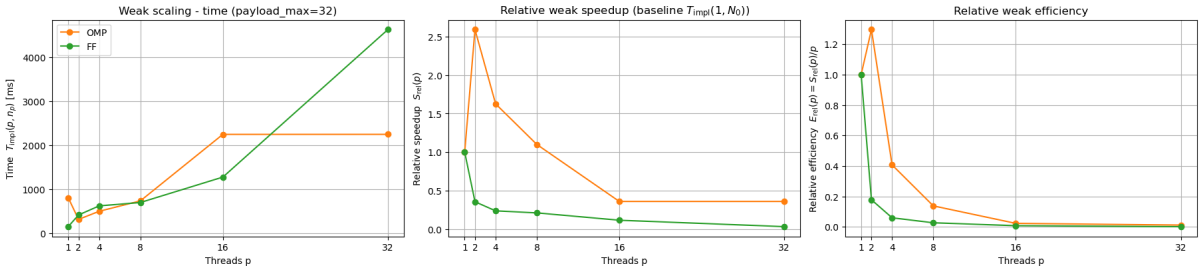

Figure 5: Weak scaling with N_p growing with p to reflect $O(N \log N)$: time, relative speedup $S_{\text{weak}}(p)$, and efficiency.

4.5 Observations

The measurements support the following points.

- **OpenMP with one thread.** The OpenMP version with a single thread is much slower than the sequential baseline. In this case I enforce a `taskwait` to ensure the reading task finishes before merging begins, which completely removes any overlap between reading and merging and likely contributes to the slowdown.
- **FastFlow with two threads.** With exactly two threads, one acts as emitter and only one remains as a worker, so this specific case cannot beat the sequential baseline. For three or more threads, the FastFlow version outperforms the sequential baseline.
- **Scaling with threads.** As I increase threads, both OpenMP and FastFlow outperform the sequential version. OpenMP is slightly better than FastFlow at higher thread counts due to lower runtime overheads.
- **Effect of problem size and payload.** Speedup grows with the number of elements and shrinks with larger payloads. Larger N increases the compute share in sorting, while larger payloads increase page faults and time spent building the index relative to sorting.
- **Memory bound cases.** With few elements and large payloads the execution tends to be memory bound. During index build, I use `mmap`, so reading one record faults in a whole page. Larger payloads reduce the number of keys per page, which increases the number of pages that need to be fetched and slows the index build.

5 Multi node implementations

I implement a hybrid MPI plus OpenMP solution with one MPI process per node and a variable number of OpenMP threads per process. Only one node touches the disk at a time.

I observed that overlapping disk activity across nodes did not improve performance on the shared Slurm cluster and that concurrent I/O from different jobs increased contention, so I serialize disk access.

5.1 Implementation overview

Rank 0 reads the unsorted file and builds the global `IndexRec` array. As soon as $\lfloor N/P \rfloor$ entries are ready, where P is the number of nodes, rank 0 partitions the index into contiguous slices and starts sending them asynchronously with `MPI_Isend`. Each receiver posts `MPI_Irecv`, sorts its local slice with OpenMP, then participates in a pairwise merge schedule. Merging proceeds in $\lceil \log_2 P \rceil$ rounds. In round r, rank i exchanges data with partner $i \oplus 2^r$ when the partner exists. The receiver merges two sorted segments into one larger sorted segment. At the end of the last round, rank 0 holds the globally sorted index, rewrites the output file, and validates correctness on the written file. No node other than rank 0 writes the output file.

5.2 Single node sorting

On each node I sort the local slice with OpenMP tasks, using the same top down mergesort as in the single node section and the same base case of 10 000 elements that falls back to `std::sort`. I keep the post-receive merge single threaded on the destination rank, both for simplicity and because each merge is short at the chunk sizes I use; the extra synchronization and cache traffic of a multithreaded merge would likely outweigh any benefit.

5.3 Performance evaluation

I analyze the MPI version in the balanced case with $\text{payload}_{\max} = 32$ bytes. The x axis is always the number of nodes. Lines in each plot denote different thread counts per node. There is exactly one MPI process per node. All times exclude the rewrite phase as discussed in Section 3.

5.3.1 Execution times over nodes

I report the total time

$$T \;=\; T_{\text{build-index}} \;+\; T_{\text{sort-index}} \;+\; T_{\text{distributed-merges}}\,,$$

for several dataset sizes N at fixed $\text{payload}_{\max} = 32$. The three panels show small, medium, and large N.

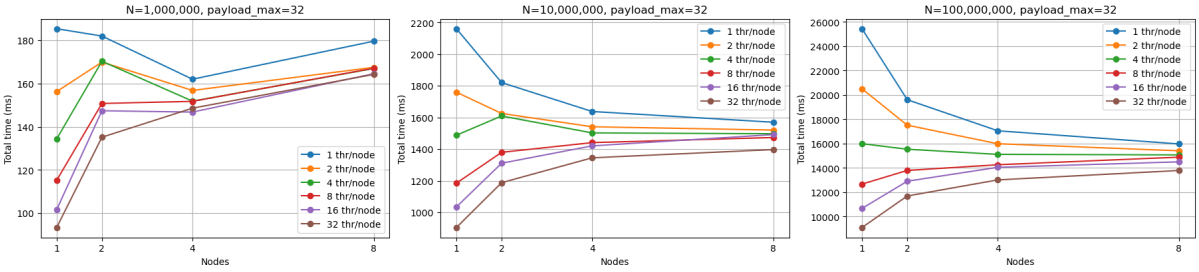

Figure 6: Execution time T versus nodes at $\text{payload}_{\max} = 32$. Panels correspond to increasing N. Lines are threads per node.

5.3.2 Absolute speedup at fixed size

Absolute speedup compares MPI performance to the best sequential baseline on the same dataset:

$$S_{\text{abs}}(P,t) \;=\; \frac{T^{*}_{\text{seq}}(N)}{T_{\text{mpi}}(P,t,N)}\,,$$

where P is nodes, t is threads per node, and $T^{*}_{\text{seq}}(N)$ is the fastest observed sequential time at size N. I also report absolute efficiency that normalizes by the number of nodes:

$$E_{\text{abs}}(P,t) \;=\; \frac{S_{\text{abs}}(P,t)}{P}\,.$$

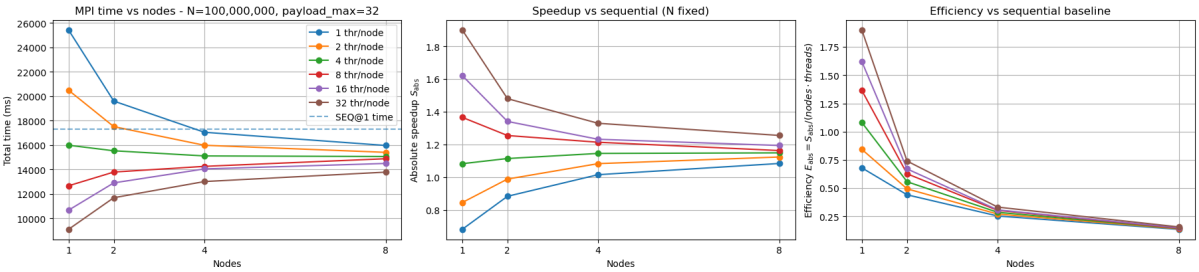

Figure 7: Fixed N and $\text{payload}_{\max} = 32$: time T, absolute speedup S_{abs}, and absolute efficiency E_{abs} versus nodes. Lines are threads per node.

5.3.3 Strong scalability

For strong scaling I fix $N = 100{,}000{,}000$ and $\text{payload}_{\max} = 32$. I define relative speedup per thread count as

$$S_{\text{strong}}(P;t) \;=\; \frac{T_{\text{mpi}}(1,t,N)}{T_{\text{mpi}}(P,t,N)}\,, \qquad E_{\text{strong}}(P;t) \;=\; \frac{S_{\text{strong}}(P;t)}{P}\,.$$

Each curve uses its own one node baseline at the same t.

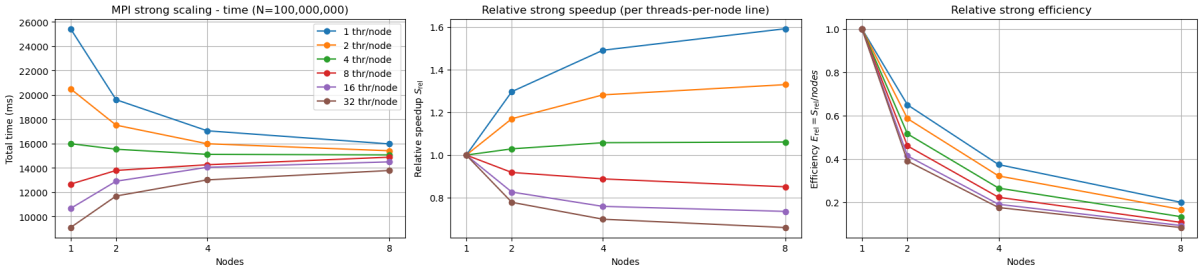

Figure 8: Strong scaling at $N = 100{,}000{,}000$ and $\text{payload}_{\max} = 32$: time, relative speedup S_{strong}, and efficiency E_{strong} versus nodes.

5.3.4 Weak scalability

For weak scaling I increase the number of elements with the number of nodes to reflect the $O(N \log N)$ cost of sorting. I choose sizes N_P so that $N_P \log_2 N_P$ grows approximately linearly with P. Relative speedup and efficiency are

$$S_{\text{weak}}(P;t) \;=\; \frac{T_{\text{mpi}}(1,t,N_1)}{T_{\text{mpi}}(P,t,N_P)}\,, \qquad E_{\text{weak}}(P;t) \;=\; \frac{S_{\text{weak}}(P;t)}{P}\,.$$

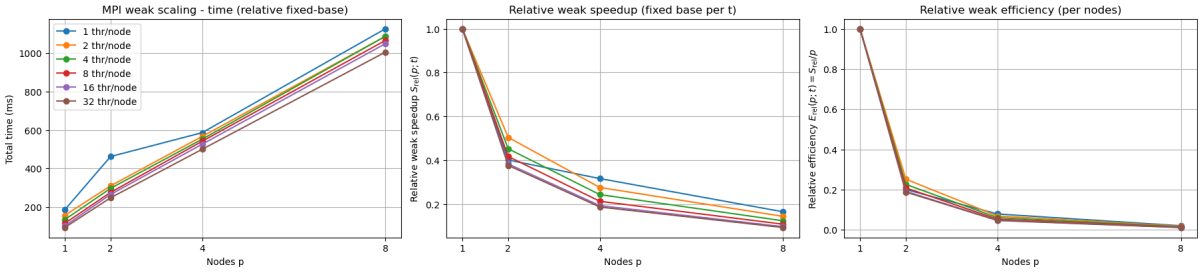

Figure 9: Weak scaling with $\text{payload}_{\max} = 32$ and N_P chosen so that $N_P \log_2 N_P$ scales with P: time, relative speedup S_{weak}, and efficiency E_{weak}.

5.4 Observations

The results show the following trends.

- **Few threads per node.** With one or a few threads per node I gain by distributing across nodes. Communication costs are small relative to local sorting time, so adding nodes reduces time meaningfully.
- **Many threads per node.** As I increase threads per node, local sorting becomes faster and the communication share grows. Beyond a point, spreading work across nodes hurts. For example, with 32 threads per node the communication time outweighs the benefit of splitting work across 8 nodes, making the run communication bound.

- **Communication-bound crossover.** As on-node sorting speeds up, the fraction of time spent in communication grows. Message startup and bandwidth transfer for sending and receiving index segments dominate, so the execution becomes communication bound.
- **Pairwise merge underutilization.** The pairwise merge schedule inherently halves the number of active ranks each round, which leaves increasing fractions of the cluster idle during merging as P grows. This structural underutilization is visible as a decreasing efficiency with more nodes in the strong and weak scaling plots.

6 Approximate cost model

This section proceeds in four parts. First, I define the variables and assumptions. Second, I present a BSP like, row-by-row sketch of the distributed pipeline to make the supersteps explicit. Third, I derive asymptotic formulas for the cost of each superstep as functions of the number of records N and the number of nodes P. Finally, I summarize the main observations that connect the model with the measurements.

6.1 Variables and assumptions

I use the following symbols and assumptions.

- N: number of records. P: number of nodes (one MPI process per node). t: threads per node.
- F: input file size in bytes. It depends on keys and payloads.
- s_I: size in bytes of one `IndexRec` entry $\{\texttt{key}, \texttt{pos}, \texttt{len}\}$ on this platform.
- $I = N \cdot s_I$: total size in bytes of the in memory index array.
- B_{read}, B_{write}: effective disk read and write bandwidths.
- B_{net}: effective per process network bandwidth for index transfers.
- c_s: constant for local comparison based sorting, so sorting M keys costs $c_s\, M \log_2 M$.
- c_m: constant for linear merges, so merging two runs of total length M costs $c_m\, M$.

All times reported in plots exclude the rewrite phase, but I keep a write term in the model for completeness.

6.2 BSP representation of the program

The program proceeds through logical stages that suggest a BSP like view: rank 0 reads and builds the index while scattering slices, all ranks sort locally, ranks participate in $\lceil \log_2 P \rceil$ merge rounds, then rank 0 rewrites the output. The figure is schematic and highlights the critical path.

6.3 BSP representation of the program

The stages are: read and build the global index on rank 0, send index chunks with asynchronous `Isend`, local sorts on all ranks, receive-and-merge rounds, then the final write on rank 0. For clarity I illustrate $P = 4$.

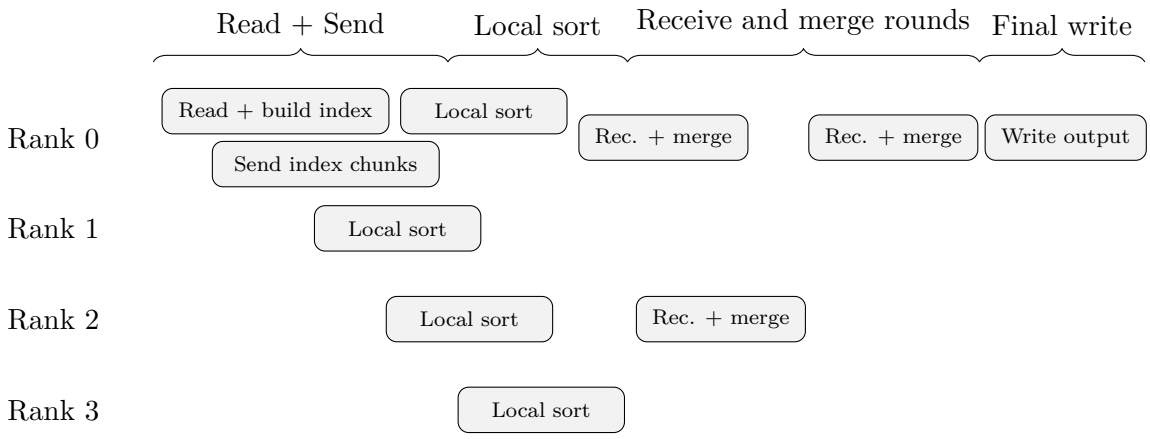

Figure 10: BSP like sketch with one row per rank (example with $P = 4$). Rank 0 overlaps reading and sending; local sorts start earlier on ranks 0, 1, and 2 than on rank 3; the first receive-and-merge round runs only on ranks 0 and 2, then the final round runs only on rank 0, followed by the final write.

6.4 Asymptotic analysis of the computations

I now write node aware expressions that capture scaling in N and P.

Read and scatter. Rank 0 reads the file and scatters the index. Sends are posted sequentially, so the scatter time is proportional to I. Reading and scattering overlap only partially, so I model the stage as

$$T_{\text{read+scatter}}(P) \;=\; \max\Big\{\frac{F}{B_{\text{read}}},\; \frac{I}{B_{\text{net}}}\Big\}.$$

Local sort. Each rank sorts about N/P keys. The cost is

$$T_{\text{sort}}(P) \;=\; c_s\,\frac{N}{P}\,\log_2\!\Big(\frac{N}{P}\Big).$$

Multi round merges and transfers. Pairwise merging proceeds in $\lceil \log_2 P \rceil$ rounds. Each round performs a linear merge of total length N along the critical path, plus transfers of index segments. Ignoring the shrinking message sizes in later rounds gives an upper bound

$$T_{\text{merge+collect}}(P) \;=\; \Big(c_m\,N + \frac{I}{B_{\text{net}}}\Big)\,\log_2 P.$$

Final write. Rank 0 streams from the unsorted mapping and writes the output:

$$T_{\text{write}} \;=\; \frac{F}{B_{\text{read}}} + \frac{F}{B_{\text{write}}}.$$

Total. The total predicted time is

$$T_{\text{total}}(P) \;=\; T_{\text{read+scatter}}(P) \;+\; T_{\text{sort}}(P) \;+\; T_{\text{merge+collect}}(P) \;+\; T_{\text{write}}.$$

6.5 Observations

- Increasing P reduces the local sorting cost $c_s\,(N/P)\log_2(N/P)$, which favors large P when single node sorting is the bottleneck.
- Communication and the number of merge rounds grow with $\log_2 P$. When the network is slow or per round merges are expensive, small P is preferable.
- The tradeoff above matches the measurements: with few threads per node I gain by distributing across nodes, while with many threads per node communication dominates and adding nodes hurts.
- Thanks to the `IndexRec` vector, computation and communication terms are independent of the payload size and scale as $N \log N$ in N and as $\log P$ in P. The disk bound read and write terms are linear in N but proportional to the file size F, which grows with the maximum payload, so I/O is sensitive to $\text{payload}_{\max}$.